MINISTRY OF EDUCATION, CULTURE AND RESEARCH OF THE REPUBLIC OF MOLDOVA
Technical University of Moldova
Faculty of Computers, Informatics and Microelectronics
Department of Software Engineering and Automation

Ion Dulgheru, group _____

Report

Practical Work No. 7
Operating Systems

Topic: **Processes and the System-Call Interface (Linux)**

Checked by:
_____, university assistant
DISA, FCIM, UTM

Chişinău – 2026

Table of Contents

1 AIM OF THE WORK..... 3
1.1 Purpose of practical work..... 3
1.2 Problem statement..... 3
2 THEORETICAL BACKGROUND..... 4
2.1 Processes and system calls..... 4
2.2 Creation, replacement and termination of processes..... 4
2.3 File descriptors and redirection..... 4
2.4 Pipes and standard I/O buffering..... 4
3 DESCRIPTION OF EXERCISES..... 5
3.1 Environment preparation..... 5
3.2 Part 1. Tracing the real shell with strace..... 5
3.3 Part 2. Process creation, zombie process, quiz and buffer..... 7
3.4 Part 3. The program myrun: fork, exec and wait..... 10
3.5 Part 4. Redirection between fork and exec..... 12
3.6 Part 5. A pipeline implemented by hand..... 13
CONCLUSION..... 16
REFERENCES..... 17

1 AIM OF THE WORK

1.1 Purpose of practical work

The purpose of the practical work is to study how an operating system creates and manages processes through the system-call interface, using Linux as the reference platform [1]. A command shell is an ordinary program built from a small set of system calls: fork, exec, wait, exit, open, dup2 and pipe [2].

In the first stage, the system calls issued by the real bash shell are observed with strace. In the second stage, the same mechanisms are implemented in C, so that the resulting programs form the skeleton of a simple shell.

1.2 Problem statement

The practical work consists of five consecutive tasks:

- tracing the system calls of the pipeline ls / | wc -l with strace;
- creating processes with fork, collecting the exit status with waitpid and observing a zombie process;
- implementing the program myrun, which executes a given command, waits for it and reports how it ended;
- implementing the redirection of standard output to a file, equivalent to ls -l > out.txt;
- implementing the pipeline ls -l /etc | wc -l with pipe, two forks, dup2 and exec.

Every program is compiled with gcc using the options -Wall -Wextra, and every warning must be fixed before execution. The results are compared with the behaviour of the real shell, and the questions of the assignment are answered on the basis of the observed output.

2 THEORETICAL BACKGROUND

2.1 Processes and system calls

A process is a running instance of a program. Each process has its own address space, its own table of open file descriptors and a unique process identifier (PID) [3]. A user program cannot access hardware or kernel data directly; it requests kernel services through system calls.

The tool strace intercepts and prints every system call made by a program together with its arguments and return value [4]. With the option -f the calls of all child processes are traced as well, and with the option -c a summary table is printed instead of the individual calls.

2.2 Creation, replacement and termination of processes

The call fork creates a new process that is a copy of the calling one; it returns 0 in the child, the PID of the child in the parent and -1 on error [5]. On Linux the glibc function fork is a wrapper over the system call clone, which is why strace shows clone (or clone3) and not fork [6].

The family of exec functions replaces the program image of the calling process with a new program [7]. After a successful call the old code no longer exists in the process, therefore exec returns only on failure. The variant execvp searches the directories listed in PATH.

When a process terminates, the kernel keeps its exit status in the process table until the parent reads it with wait or waitpid [8]. A terminated child whose status has not yet been collected is a zombie process, marked with the state Z. The macros WIFEXITED, WEXITSTATUS, WIFSIGNALED and WTERMSIG decode the status value.

2.3 File descriptors and redirection

Each process starts with three open descriptors: 0 (standard input), 1 (standard output) and 2 (standard error). The call dup2(old, new) makes the descriptor new refer to the same open file as old [9]. The descriptor table is copied by fork and preserved by exec, therefore a shell performs the redirection in the child process, between fork and exec.

2.4 Pipes and standard I/O buffering

The call pipe creates a unidirectional channel and returns two descriptors: p[0] for reading and p[1] for writing [10]. A read from the pipe returns 0 (end of file) only when all descriptors of the write end, in all processes, are closed. After fork each process holds its own copy of both descriptors, so every unused end has to be closed.

The standard I/O library keeps the text of printf in a buffer located in the memory of the process. The buffer is line-buffered when the output is a terminal and fully buffered when the output is a file or a pipe [11]. Since fork copies the whole memory of the process, an unflushed buffer is duplicated as well.

3 DESCRIPTION OF EXERCISES

3.1 Environment preparation

The work was performed on a laptop with Ubuntu Linux. The compiler gcc 15.2.0 and the tracer strace 6.19 were already installed, and the working directory ~/os-lab7 was created together with the file report.md, as shown in figure 3.1.

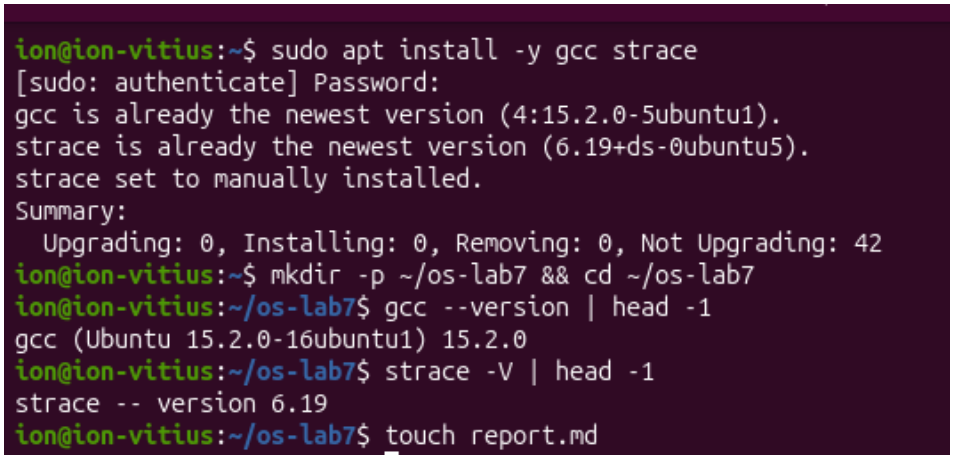


Figure 3.1 – Installation check and creation of the working directory

All programs of this work were compiled with the command gcc -Wall -Wextra -o NAME NAME.c, and no warning was reported.

3.2 Part 1. Tracing the real shell with strace

The two-stage pipeline executed by bash was traced with the following commands, followed by the counting of the system calls of ls:

```
strace -f -e trace=pipe2,clone,clone3,dup2,execve,wait4 bash -c 'ls / | wc -l'
strace -c ls /usr/bin > /dev/null
strace -c -e trace=write ls /usr/bin > /dev/null
```

The output of the first command is shown in figure 3.2. The order of the calls and the processes that issue them are summarized in table 3.1.

```
ion@ion-vitius:~/os-lab7$ touch report.md
ion@ion-vitius:~/os-lab7$ strace -f -e trace=pipe2,clone,clone3,dup2,execve,wait4 bash -c 'ls / | wc -l'
execve("/usr/bin/bash", ["bash", "-c", "ls / | wc -l"], 0x7ffc9b8aea78 /* 61 vars */) = 0
pipe2([3, 4], 0)                        = 0
clone(child_stack=NULL, flags=CLONE_CHILD_CLEARTID|CLONE_CHILD_SETTID|SIGCHLD, child_tidptr=0x73605a914a10) = 12645
strace: Process 12645 attached
[pid 12641] clone(child_stack=NULL, flags=CLONE_CHILD_CLEARTID|CLONE_CHILD_SETTID|SIGCHLD, child_tidptr=0x73605a914a1
) = 12646
strace: Process 12646 attached
[pid 12645] dup2(4, 1)                  = 1
[pid 12641] wait4(-1 <unfinished ...>
[pid 12646] dup2(3, 0)                  = 0
[pid 12645] execve("/usr/bin/ls", ["ls", "/"], 0x55d8b10ecb70 /* 61 vars */ <unfinished ...>
[pid 12646] execve("/usr/bin/wc", ["wc", "-l"], 0x55d8b10ecb70 /* 61 vars */ <unfinished ...>
[pid 12645] <... execve resumed>)       = 0
[pid 12646] <... execve resumed>)       = 0
[pid 12645] +++ exited with 0 +++
23
[pid 12641] <... wait4 resumed>, [{WIFEXITED(s) && WEXITSTATUS(s) == 0}], 0, NULL) = 12645
[pid 12641] wait4(-1 <unfinished ...>
[pid 12646] +++ exited with 0 +++
<... wait4 resumed>, [{WIFEXITED(s) && WEXITSTATUS(s) == 0}], 0, NULL) = 12646
--- SIGCHLD {si_signo=SIGCHLD, si_code=CLD_EXITED, si_pid=12645, si_uid=1000, si_status=0, si_utime=0, si_stime=0} --
wait4(-1, 0x7ffdfb18e354, WNOHANG, NULL) = -1 ECHILD (No child processes)
+++ exited with 0 +++
```

Figure 3.2 – System calls of the pipeline ls / | wc -l

Table 3.1 – Order of the system calls in the pipeline ls / | wc -l

Step	PID	System call	Purpose
1	12641 (bash)	pipe2([3, 4], 0)	creates the pipe: read end 3, write end 4
2	12641 (bash)	clone(...) = 12645	creates the child for ls
3	12641 (bash)	clone(...) = 12646	creates the child for wc
4	12645	dup2(4, 1)	standard output of ls goes to the pipe
5	12646	dup2(3, 0)	standard input of wc comes from the pipe
6	12645	execve("/usr/bin/ls", ...)	the child becomes ls
7	12646	execve("/usr/bin/wc", ...)	the child becomes wc
8	12641 (bash)	wait4(-1, ...)	the shell collects both children

According to table 3.1, the sequence is pipe2, clone, clone, dup2, execve, wait4. The process 12645 calls dup2(4, 1) because ls always writes to descriptor 1, and the redirection makes this descriptor refer to the write end of the pipe instead of the terminal. The process 12646 calls dup2(3, 0) because wc always reads from descriptor 0, and the redirection connects this descriptor to the read end of the pipe, so that wc receives the output of ls. The value 23 printed on the terminal is the number of entries of the root directory counted by wc.

The trace contains no line named fork. The children are created by the system call clone, with the flags CLONE_CHILD_CLEARTID, CLONE_CHILD_SETTID and SIGCHLD, because the glibc function fork is implemented over clone.

The summary of all system calls made by ls /usr/bin is presented in figure 3.3. The command made 161 calls in total, 10 of which ended with an error, but the second command shows that only 3 of them are write calls.

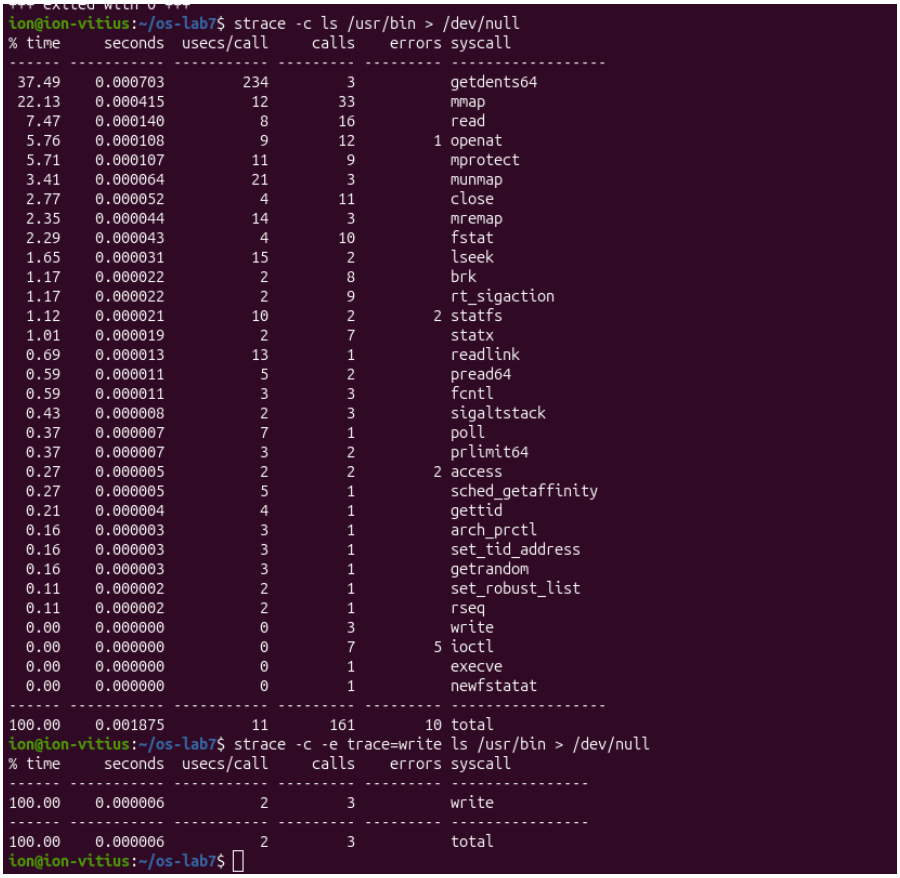


Figure 3.3 – Summary of the system calls of ls /usr/bin

The number of write calls is small because the output of ls is redirected to /dev/null, and in this case the standard output is fully buffered. The C library accumulates the text in a buffer of block size and writes it with one call when the buffer is full, instead of one call per line. The most calls belong to the loading of the program and of the libraries (mmap, openat, mprotect), not to the output.

3.3 Part 2. Process creation, zombie process, quiz and buffer

The program fork.c creates one child that exits with the code 42, while the parent collects the status with waitpid:

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
int main(void)
{
    printf("before fork: pid=%d\n", getpid());
    fflush(stdout);
    pid_t pid = fork();
    if (pid < 0) { perror("fork"); exit(1); }
    if (pid == 0) {                          /* child */
        printf("child: pid=%d parent=%d\n", getpid(), getppid());
        exit(42);
    }
    int status;                              /* parent */
    waitpid(pid, &status, 0);
    printf("parent: pid=%d, my child %d exited with %d\n",
           getpid(), pid, WEXITSTATUS(status));
    return 0;
}
```

The result of the execution is shown in figure 3.4. The child (PID 12804) reports the parent 12803, and the parent receives the status 42 that was passed to exit in the child.

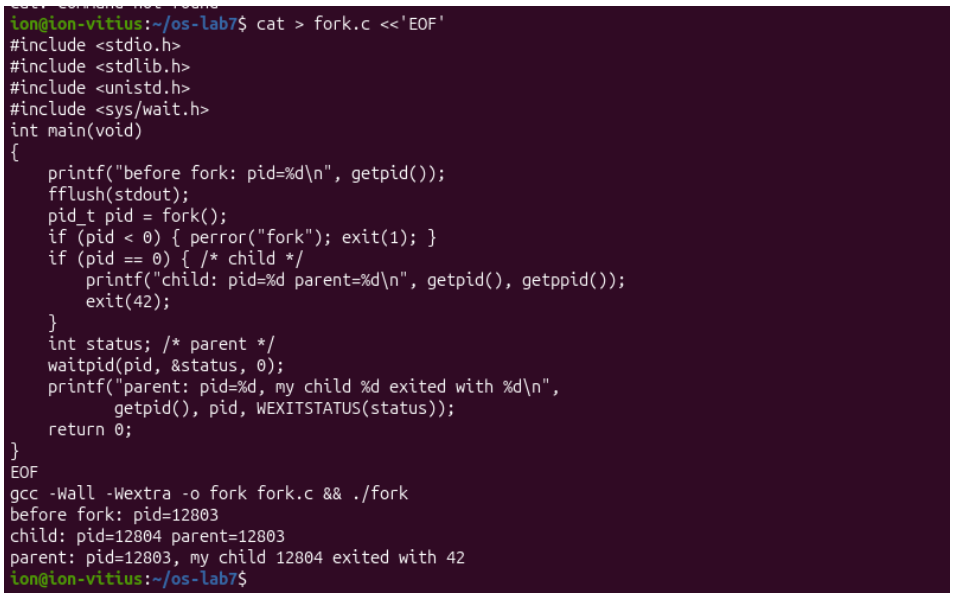


Figure 3.4 – Execution of fork.c

The zombie process was obtained by copying fork.c to zombie.c and inserting the call sleep(30); immediately before waitpid in the parent:

```c
    sleep(30);   /* the parent does not wait yet: the child is a zombie */
    waitpid(pid, &status, 0);
```

While the parent sleeps, the table of processes shows the child in the state Z, as presented in figure 3.5. The child 12838 of the parent 12836 has finished, but its entry remains in the table and is displayed as [zombie] <defunct>.

```
ion@ion-vitius:~/os-lab7$ cp fork.c zombie.c
sed -i 's|waitpid(pid, &status, 0);|sleep(30); waitpid(pid, \&status, 0);|' zombie.c
grep -n sleep zombie.c
gcc -Wall -Wextra -o zombie zombie.c
./zombie &
sleep 1; ps -o pid,ppid,stat,cmd --ppid $!
16:    sleep(30); waitpid(pid, &status, 0);
[1] 12836
before fork: pid=12836
child: pid=12838 parent=12836
    PID    PPID STAT CMD
  12838   12836 Z    [zombie] <defunct>
ion@ion-vitius:~/os-lab7$
```

Figure 3.5 – Zombie process while the parent sleeps

After 30 seconds the parent calls waitpid and collects the exit status. According to figure 3.6, the same ps command prints only the header, which means that the entry of the child was removed from the table.

```
ion@ion-vitius:~/os-lab7$ sleep 31; ps -o pid,ppid,stat,cmd --ppid 12836
    PID    PPID STAT CMD
ion@ion-vitius:~/os-lab7$
```

Figure 3.6 – The table of processes after waitpid

The program quiz.c contains three consecutive fork calls followed by printf:

```
#include <stdio.h>
#include <unistd.h>
int main(void) { fork(); fork(); fork(); printf("hi\n"); return 0; }
```

Every fork call doubles the number of existing processes, and all of them reach printf. According to formula (3.1), the predicted number of lines is 2 to the power 3, that is 8:

$$N = 2^k \tag{3.1}$$

where N is the number of processes that execute printf; k is the number of fork calls executed unconditionally in sequence (here k = 3).

The measured result is shown in figure 3.7 and matches the prediction: the command ./quiz | wc -l printed 8.

```
ion@ion-vitius:~/os-lab7$ cat > quiz.c <<'EOF'
#include <stdio.h>
#include <unistd.h>
int main(void) { fork(); fork(); fork(); printf("hi\n"); return 0; }
EOF
gcc -Wall -o quiz quiz.c && ./quiz | wc -l
8
ion@ion-vitius:~/os-lab7$
```

Figure 3.7 – Execution of quiz.c

The tree of the 8 processes is drawn in figure 3.8. A process created by the k-th fork call executes only the fork calls that follow it; therefore the original process P has three children (A, B, C), the process A has two (A1, A2), and the processes A1 and B have one child each.

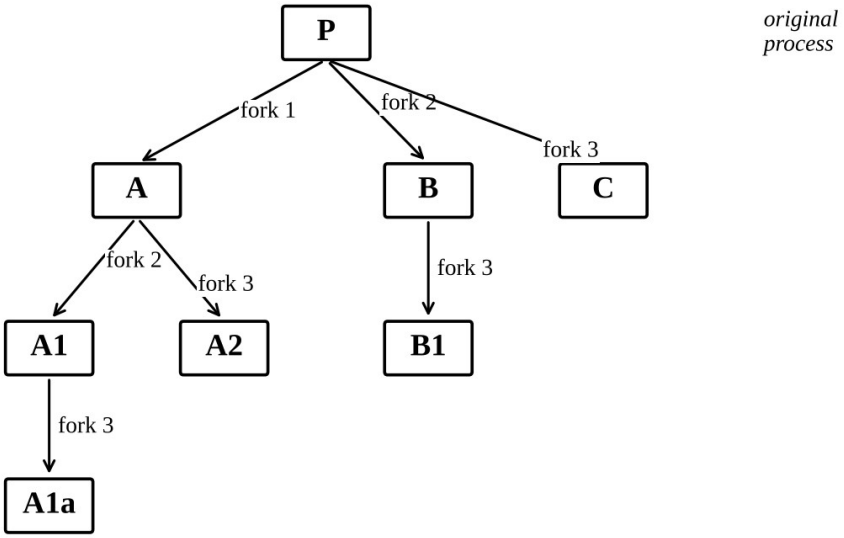


Figure 3.8 – Process tree of quiz.c

In the buffer puzzle the line fflush(stdout); was removed from fork.c, and the program was executed once on the terminal and once through a pipe. The result is presented in figure 3.9.

```
ion@ion-vitius:~/os-lab7$ sed '/fflush(stdout);/d' fork.c > fork_nobuf.c
grep -c fflush fork_nobuf.c
gcc -Wall -Wextra -o fork_nobuf fork_nobuf.c
./fork_nobuf
./fork_nobuf | cat
0
before fork: pid=12950
child: pid=12951 parent=12950
parent: pid=12950, my child 12951 exited with 42
before fork: pid=12952
child: pid=12954 parent=12952
before fork: pid=12952
parent: pid=12952, my child 12954 exited with 42
ion@ion-vitius:~/os-lab7$
```

Figure 3.9 – Execution of fork without fflush: terminal and pipe

On the terminal, the line before fork appears once. Through the pipe (PID 12952), the line before fork: pid=12952 appears twice. When the output is a terminal, stdout is line-buffered and the character \n flushes the buffer immediately, so the text is already written before fork. When the output is a pipe, stdout is fully buffered and the text remains in the buffer in the memory of the process. The call fork copies the whole memory, including this buffer, so both the parent and the child hold the same unflushed text and each of them writes it when the buffer is flushed at exit.

3.4 Part 3. The program myrun: fork, exec and wait

The program myrun.c executes the command given on the command line, waits for it and reports how it ended. It represents the main loop of a shell without the parsing of commands:

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
int main(int argc, char *argv[])
{
    if (argc < 2) {
        fprintf(stderr, "usage: %s program [args...]\n", argv[0]);
        return 1;
    }
    pid_t pid = fork();
    if (pid < 0) { perror("fork"); return 1; }
    if (pid == 0) {
        execvp(argv[1], &argv[1]);
        perror(argv[1]);
        exit(127);
    }
    int status;
    waitpid(pid, &status, 0);
    if (WIFEXITED(status))
        printf("[myrun] %s exited with status %d\n", argv[1], WEXITSTATUS(status));
    else if (WIFSIGNALED(status))
        printf("[myrun] %s killed by signal %d\n", argv[1], WTERMSIG(status));
    return 0;
}
```

All four outcomes of the assignment were tested; the results are summarized in table 3.2, and the terminal output is shown in figure 3.10.

Table 3.2 – Outcomes reported by myrun

Command	Reported result	Explanation
./myrun ls -d /etc	exited with status 0	ls finished successfully
./myrun false	exited with status 1	false always returns 1
./myrun nosuchprog	exited with status 127	execvp failed, the child called exit(127)
./myrun sleep 100 & pkill -x sleep	killed by signal 15	the child was terminated by SIGTERM

```
parent: pid=12992, my child 12994 exited with 42
ion@ion-vitius:~/os-lab7$ cat > myrun.c <<'EOF'
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
int main(int argc, char *argv[])
{
    if (argc < 2) {
        fprintf(stderr, "usage: %s program [args...]\n", argv[0]);
        return 1;
    }
    pid_t pid = fork();
    if (pid < 0) { perror("fork"); return 1; }
    if (pid == 0) {
        execvp(argv[1], &argv[1]);
        perror(argv[1]);
        exit(127);
    }
    int status;
    waitpid(pid, &status, 0);
    if (WIFEXITED(status))
        printf("[myrun] %s exited with status %d\n", argv[1], WEXITSTATUS(status));
    else if (WIFSIGNALED(status))
        printf("[myrun] %s killed by signal %d\n", argv[1], WTERMSIG(status));
    return 0;
}
EOF
gcc -Wall -Wextra -o myrun myrun.c
./myrun ls -d /etc
./myrun false
./myrun nosuchprog
./myrun sleep 100 & sleep 1; pkill -x sleep
bash -c nosuchprog; echo $?
kill -l | tr '\t' '\n' | grep -w 15
/etc
[myrun] ls exited with status 0
[myrun] false exited with status 1
nosuchprog: No such file or directory
[myrun] nosuchprog exited with status 127
[1] 12995
pkill: killing pid 2639 failed: Operation not permitted
[myrun] sleep killed by signal 15
bash: line 1: nosuchprog: command not found
[1]+  Done                    ./myrun sleep 100
127
15) SIGTERM
49) SIGRTMIN+15
ion@ion-vitius:~/os-lab7$
```

Figure 3.10 – Execution of myrun for the four outcomes

The code after execvp is executed only when exec fails. A successful exec replaces the code, data and stack of the process with the new program, so the old instructions no longer exist and the call never returns. The lines perror and exit(127) are reached only if the program was not found or could not be executed.

The value 127 is the convention of the shell for the message command not found. The command bash -c nosuchprog printed bash: line 1: nosuchprog: command not found, and echo $? printed 127, which is identical to the status reported by myrun.

The command pkill sends by default the signal 15, and kill -l shows that it is 15) SIGTERM [12]. The message pkill: killing pid 2639 failed: Operation not permitted refers to another process named sleep, owned by a different user, and does not affect the result, because the own process of the user was terminated and reported correctly.

3.5 Part 4. Redirection between fork and exec

The program redirect.c reproduces the effect of the command ls -l > out.txt. In the child, the file is opened, the descriptor 1 is replaced with dup2, the extra descriptor is closed and ls is executed; the parent only waits:

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/wait.h>
int main(void)
{
    pid_t pid = fork();
    if (pid < 0) { perror("fork"); return 1; }
    if (pid == 0) {
        int fd = open("out.txt", O_WRONLY | O_CREAT | O_TRUNC, 0644);
        if (fd < 0) { perror("open"); exit(1); }
        dup2(fd, 1);
        close(fd);
        execlp("ls", "ls", "-l", (char *)NULL);
        perror("execlp");
        exit(127);
    }
    waitpid(pid, NULL, 0);
    return 0;
}
```

According to figure 3.11, the program printed nothing on the screen, and the file out.txt contains the listing produced by ls -l. The file out.txt itself appears in the list, because it is created by open before ls reads the directory.

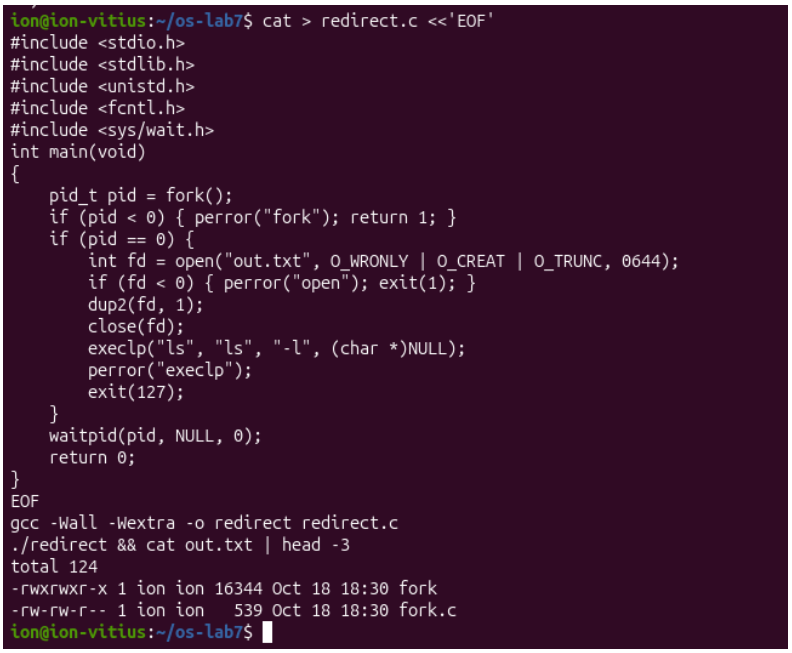


Figure 3.11 – Execution of redirect and the first lines of out.txt

3.6 Part 5. A pipeline implemented by hand

The program pipeline.c implements ls -l /etc | wc -l with one pipe, two forks, dup2 and exec. Both children close the original pipe descriptors after dup2, and the parent closes both ends before waiting:

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
int main(void)
{
```

```
    int p[2];
    if (pipe(p) < 0) { perror("pipe"); return 1; }

    pid_t a = fork();
    if (a < 0) { perror("fork"); return 1; }
    if (a == 0) {                  /* ls -l /etc -> writes into the pipe */
        dup2(p[1], 1);
        close(p[0]); close(p[1]);
        execlp("ls", "ls", "-l", "/etc", (char *)NULL);
        perror("ls"); exit(127);
    }

    pid_t b = fork();
    if (b < 0) { perror("fork"); return 1; }
    if (b == 0) {                  /* wc -l <- reads from the pipe */
        dup2(p[0], 0);
        close(p[0]); close(p[1]);
        execlp("wc", "wc", "-l", (char *)NULL);
        perror("wc"); exit(127);
    }

    close(p[0]);
    close(p[1]);                   /* ESSENTIAL: otherwise wc never gets EOF */
    waitpid(a, NULL, 0);
    waitpid(b, NULL, 0);
    return 0;
}
```

The program and the real shell command printed the same value, 246, as presented in figure 3.12.

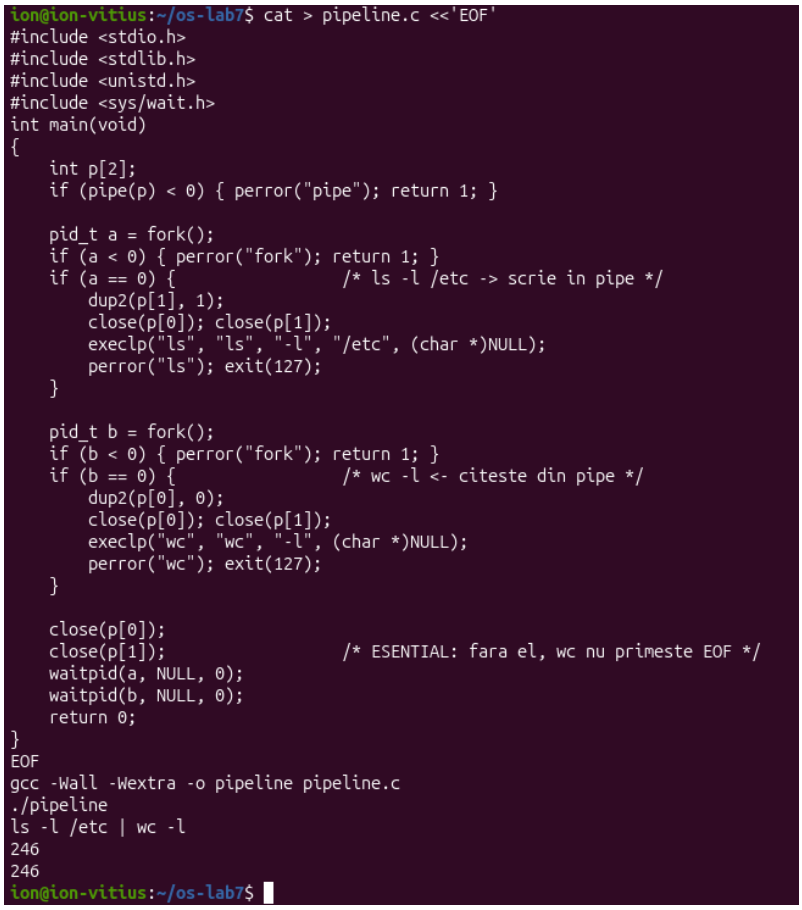


Figure 3.12 – Execution of pipeline compared with ls -l /etc | wc -l

For the experiment with the hang, the line close(p[1]); of the parent was commented out, and the program was recompiled as pipeline_hang:

```
    close(p[0]);
    /* close(p[1]); */
```

After this change the program no longer returned to the prompt and had to be stopped with Ctrl-C. The descriptors of both processes were inspected from a second terminal with ls -l /proc/<pid>/fd, and the result is shown in figure 3.13. The process wc has the descriptor 0 connected to pipe:[65157] in read mode, and the parent pipeline_hang still has the descriptor 4 connected to the same pipe:[65157] in write mode.



Figure 3.13 – Descriptors of wc and of pipeline_hang in the blocked state

The process wc does not finish because it reads from the pipe until it receives the end of file, and the end of file is delivered only when all descriptors of the write end are closed. The parent kept p[1] open, so the kernel considered that data could still arrive, and the read of wc never returned 0. The parent waited for wc in waitpid, so the two processes blocked each other.

The redirection must be performed by the shell in the child process and not in the shell itself, because dup2 changes the descriptor table of the process that calls it. If the shell redirected its own standard output, it would lose the connection to the terminal and would have to restore it after each command. In the child, between fork and exec, the change affects only the program that is about to run, and the shell remains unchanged.

CONCLUSION

In this practical work the system-call interface of Linux was studied by observing the real shell and by implementing its mechanisms. The trace of the pipeline ls / | wc -l confirmed the sequence pipe2, clone, dup2, execve, wait4, and the creation of children through clone, which is the implementation of fork in glibc [5], [6].

The programs fork.c, zombie.c, quiz.c, myrun.c, redirect.c and pipeline.c were compiled with -Wall -Wextra and produced the expected results. The zombie state Z was observed until the parent called waitpid [8], the quiz printed 8 lines in accordance with formula (3.1), and myrun reported the statuses 0, 1, 127 and the signal 15.

The experiments explained two typical sources of errors in systems programming. Unflushed standard I/O buffers are duplicated by fork when the output is a pipe, and an unclosed write end of a pipe prevents the reader from receiving the end of file. The redirection with dup2 between fork and exec and the connection of two programs with pipe [10] represent the core of the implementation of a shell.

REFERENCES

[1] Operating Systems 2026. Lab 7: Processes and the System-Call Interface (Linux). Technical University of Moldova, FCIM, 2026.

[2] Kerrisk M. The Linux Programming Interface. San Francisco: No Starch Press, 2010. 1556 p. ISBN 978-1-59327-220-3.

[3] Tanenbaum A. S., Bos H. Modern Operating Systems. 4th ed. Boston: Pearson, 2015. 1136 p. ISBN 978-0-13-359162-0.

[4] strace(1) – Linux manual page. Available: https://man7.org/linux/man-pages/man1/strace.1.html (accessed 18.10.2026).

[5] fork(2) – Linux manual page. Available: https://man7.org/linux/man-pages/man2/fork.2.html (accessed 18.10.2026).

[6] clone(2) – Linux manual page. Available: https://man7.org/linux/man-pages/man2/clone.2.html (accessed 18.10.2026).

[7] execve(2) – Linux manual page. Available: https://man7.org/linux/man-pages/man2/execve.2.html (accessed 18.10.2026).

[8] wait(2) – Linux manual page. Available: https://man7.org/linux/man-pages/man2/wait.2.html (accessed 18.10.2026).

[9] dup(2) – Linux manual page. Available: https://man7.org/linux/man-pages/man2/dup.2.html (accessed 18.10.2026).

[10] pipe(2) – Linux manual page. Available: https://man7.org/linux/man-pages/man2/pipe.2.html (accessed 18.10.2026).

[11] setbuf(3) – Linux manual page. Available: https://man7.org/linux/man-pages/man3/setbuf.3.html (accessed 18.10.2026).

[12] signal(7) – Linux manual page. Available: https://man7.org/linux/man-pages/man7/signal.7.html (accessed 18.10.2026).